CS498 HW3 Design - Man Wai Lam(mwlam2)

Part 1: Wide-Column Schema Design

1.1 Row Key & Sharding

My proposed row key is City#Make#DOLVehicleID. The reason is starting the key with City instead of something like Model Year avoids hotspotting. If the key starts with Model Year, all the new data will be inserted to the tail of the table, so one node has to handle all the new writes. With City first, new writes go to different cities, so they spread across different nodes.

Querying by City is fast because all vehicles in the same city are stored next to each other, so it is just a range scan on the City prefix. But if we want to find vehicles by make, it will be slower, since the same make is spread across all the cities and need to scan the whole table.

DOL Vehicle ID is added at the end to make sure every row key is unique. The dataset only has first 10 the characters of the VIN, which can be the same for two cars, so if two cars in the same city had the same key, one would overwrite the other.

1.2 Column Families

I would split the columns into three column families:

vehicle: VIN (1-10), Model Year, Make, Model, Electric Vehicle Type, Electric Range

location: County, City, State, Postal Code, Vehicle Location,

Legislative District, 2020 GEOID - admin: DOL Vehicle ID, CAFV Eligibility, Electric Utility

Grouping the columns this way helps read performance because Bigtable stores each column family together on disk. So a query only needs to read the families it uses and can skip the rest.

1.3 Hashed vs Range-Based Shard Key

A hashed shard key on VIN spreads the data evenly across nodes, so new writes don't pile up on one node and there is no hotspot. Looking up one specific vehicle is also fast,

since we can compute the hash and go straight to the right node. But range queries are slow, because the data is scattered everywhere.

A range-based shard key on Model Year is good for range queries, since cars from the same years are stored together, so a query like 2020 to 2023 only needs to hit one or two nodes. But it causes hotspotting, because all the newly registered cars have the latest model year, so they all go to the last node while the other nodes stay idle.

Part 4: Performance Analysis
4.1 Latency Results

Measured with 500 requests each, from the VM

Endpoint	**Write Concern**	**Avg Latency**
/insert-fast	w=1	18.7 ms
/insert-safe	w="majority"	20.1 ms

insert-safe is about 1.4 ms slower, because it has to wait for a secondary to replicate the write before it returns. The gap is small because the three Atlas nodes are all in the same region, so replication between them is very fast. Most of the 18 ms is the network round trip from the VM to Atlas, which both endpoints pay.

4.2 CAP Scenario

One example is collecting telemetry from a sim racing rig. It sends data like 60 times per second, so there are a lot of writes and they need to be fast. If we lose one frame because the primary crashes, it's not a big deal, since the frames before and after it are almost the same and the lap analysis will still be fine. In my test, w=1 was only 1.4 ms faster, but that's because all three nodes are in the same region. If the replicas were in different regions, waiting for the majority could add tens of ms to every write, and at 60 writes per second the writes could fall behind the incoming data. So here I would pick availability and low latency over consistency, and use w=1.